CCS4202

AERIAL RELIEF AIRDROP OPTIMIZATION

USING DYNAMIC PROGRAMMING (0/1 KNAPSACK)

PRESENTED BY: GROUP 11

LOGITH A/L MOHANA KRISHNAN	223021
SASHVHANT A/L VADEVELL	223726
HASSAN SAED	213870
XU CHENHUI	217944
QIAN YIJIA	225842

GEOGRAPHICAL SETTING

REMOTE ISLAND SECTOR

On the mainland, a central Supply Hub holds critical emergency resources, while the target destination is an Isolated Island Drop Zone. These two points are separated by a highly dangerous, stormy sea.

STATUS: Port Destroyed | Sea Routes: OFFLINE
ACTIVE VECTOR: Helicopter Airdrop Only

CRISIS TIMELINE

PHASE 01

Typhoon Hits
the remote island

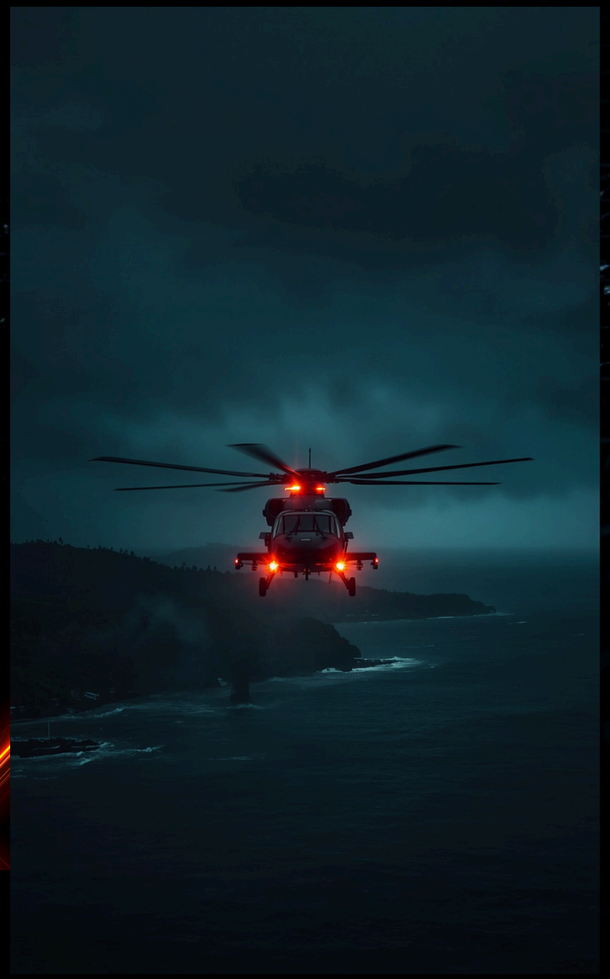
PHASE 02

typhoon completely destroyed the island's local seaport, all maritime access and ground transport routes are entirely severed

PHASE 03

helicopter airdrop as the only method to deliver supplies

THE OBJECTIVE



Our objective is to maximize the total survival priority value of the selected, indivisible supply crates without exceeding the rescue helicopter's strict maximum payload capacity of 1000 kg, *W*.

$$\max \sum_{i=1}^{n} v_i x_i$$

v_i = Survival priority value of crate i

CAPACITY CONSTRAINT

$$\sum_{i=1}^{n} w_i x_i \leqslant W$$

w_i = Weight of crate i in kg
W = Maximum aircraft payload (1000 kg)

THE 0/1 CONSTRAINT

$$x_i \in \{0, 1\}$$

x_i = 1 (Crate is loaded)
x_i = 0 (Crate is left behind)

AVAILABLE SUPPLY CRATES

CRATE MANIFEST — REMOTE ISLAND AIRDROP MISSION
1 Water Purifiers 600 kg | Value: 500
2 Emergency Rations 500 kg | Value: 450
3 Medical Supplies 500 kg | Value: 400
4 Surgical Equipment 400 kg | Value: 350
5 First Aid Kits 100 kg | Value: 150
6 Comms Gear 100 kg | Value: 120
MAX PAYLOAD: 1000 KG

Item Index (i)	Crate Content	Weight (wi) in kg	Priority Value (vi)
0	Heavy Surgical Equipment	600	800
1	High-Yield Water Purifiers	500	650
2	Concentrated Emergency Rations	500	650
3	Portable Diesel Generators	400	500
4	First Aid Trauma Kits	100	150
5	Satellite Communication Gear	100	120

WHY DYNAMIC PROGRAMMING?

× GREEDY

- Fails on 0/1 constraint
- Wasted space if high-value items don't fit
- Produces suboptimal solutions

√ DYNAMIC PROGRAMMING

- Naturally handles the 0/1 constraint via binary decision (take/leave)
- Guarantees the absolute optimal survival value
- Systematic evaluation with mathematical proof

WHY DYNAMIC PROGRAMMING?

× BRUTE FORCE

- Possible subset combinations (2^n)
- In this scenario (n=6): only 64 combinations are possible.
- In the extended scenario (n=100): 2^{100} is completely infeasible.
- The optimal solution, but at a very high cost.

√ DYNAMIC PROGRAMMING

- Overlapping subproblems are reused by filling in a table.
- In this scenario (n=6, W=1000): only 6,000 states (microseconds).
- Extended scenario (n=100, W=1000): 100,000 states (milliseconds)
- The optimal solution with minimal cost.

WHY DYNAMIC PROGRAMMING?

1.Step-by-step Greedy

Sort by Ratio (Value/kg):
Trauma Kits (1.50) → Surgery (1.33)
→Purifiers (1.30) → Rations (1.30) →
Generators (1.25) → Comms (1.20)

2.Greedy Packing Process

Pick (100kg, 150 value). Remaining: 900kg.
Pick (600kg, 800 value). Remaining: 300kg.
(500kg) too heavy. (500kg) too heavy. (400kg) too heavy.
Pick (100kg, 120 value). Remaining: 200kg (stuck).

3.Greedy Final Result

Total Weight = 800 kg (20% wasted capacity)
Total Value = 1070

DP Optimal Result

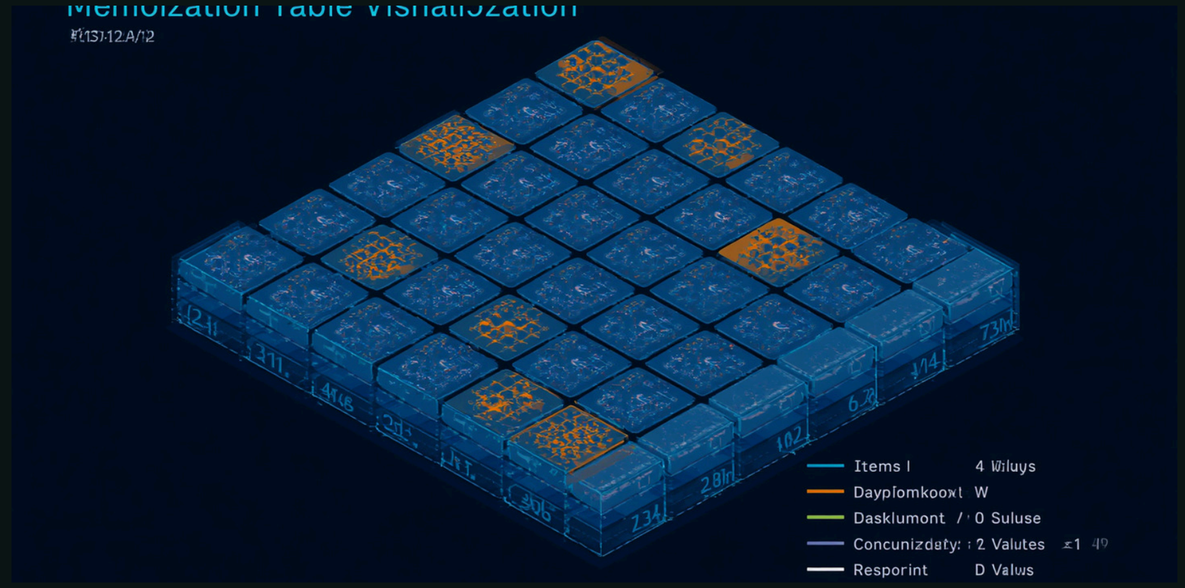
Select Water Purifiers (500kg) + Rations (500kg)
Total Weight = 1000 kg (FULL CAPACITY)
Total Value = 1300

THE COST OF GUESSING

- Value Lost: 1300 (Optimal) – 1070 (Greedy) = 230 Survival Priority Points
- Capacity Wasted: Greedy only used 800kg out of 1000kg → 20% of helicopter space wasted
- Real-World Impact: 230 points represent critical medicine, extra fuel, or communication gear that would not reach the isolated island.

Metric	Greedy (Ratio-Based)	Dynamic Programming
Total Survival Value	1070	1300 (Optimal)
Total Weight Used (kg)	800	1000
Capacity Utilization	80%	100%
Capacity Wasted	20% (200 kg)	0% (0 kg)
Items Selected	{#4, #0, #5}	{#1, #2}
Selected Crates	Trauma Kits + Surgical Equipment + Comms Gear	Water Purifiers + Rations
Result	Suboptimal	Optimal

HOW THE DP ALGORITHM THINKS



OVERLAPPING SUBPROBLEMS

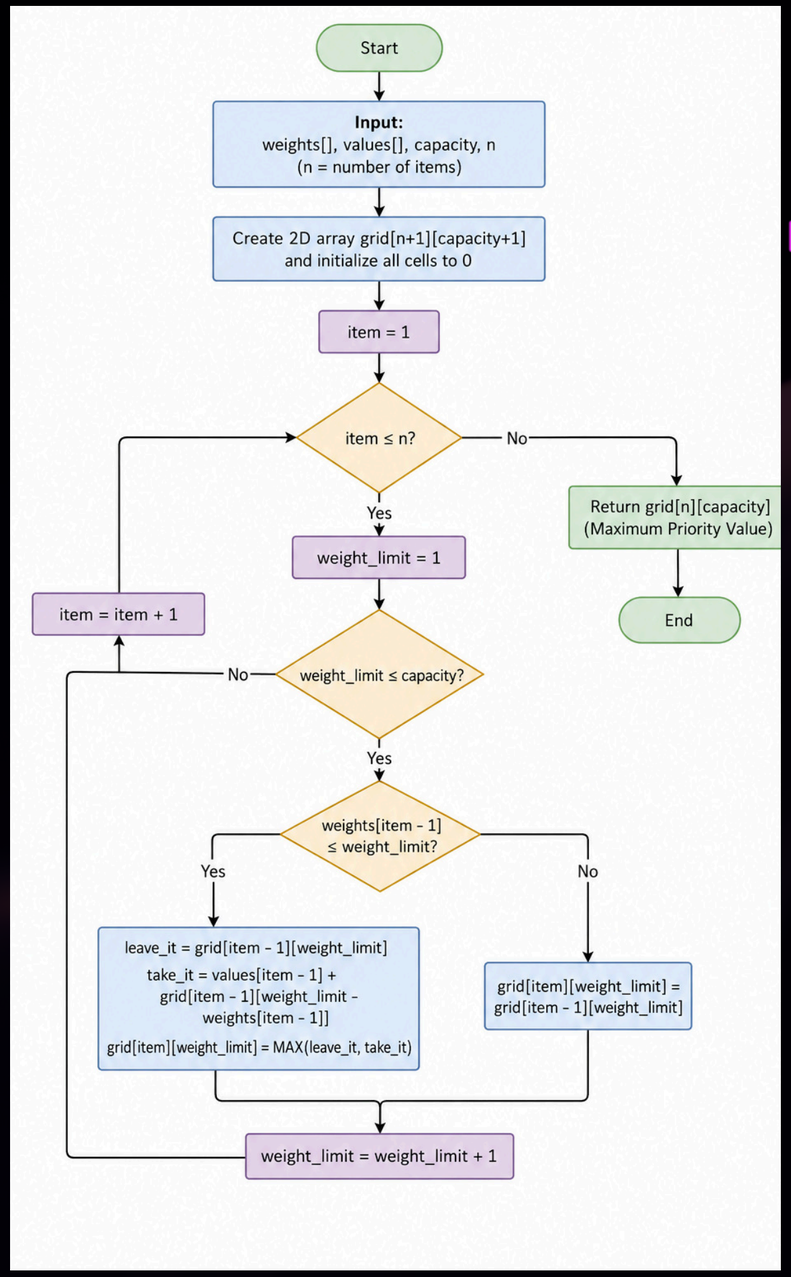
Solutions for smaller sub-capacities are computed once and reused. Each smaller weight solution builds systematically up to the full 1000 KG capacity.

MATRIX TABLE

Best value combinations stored in a 2D table DP[i][w]. Avoids redundant recalculations, each cell solved exactly once for O(n·W) efficiency.

INIT ITERATE RESOLVE

EXECUTION FLOW



THE FLOWCHART VISUALLY TRACKS HOW THE SYSTEM SOLVES THE PROBLEM STEP-BY-STEP USING A NESTED LOOP STRUCTURE. IT BEGINS BY CREATING A BLANK 2D TABLE WHERE ROWS REPRESENT ITEMS AND COLUMNS REPRESENT WEIGHT BUDGETS FROM 1 TO 1000 KG. THE CHART THEN LOOPS CELL-BY-CELL, USING A DECISION DIAMOND TO CHECK IF AN ITEM FITS THE CURRENT WEIGHT LIMIT. IF IT IS TOO HEAVY, THE FLOW COPIES THE BEST SCORE FROM THE ROW DIRECTLY ABOVE; IF IT FITS, THE PATH SPLITS TO PICK THE HIGHER VALUE BETWEEN LEAVING THE ITEM BEHIND OR TAKING IT AND ADDING IT TO WHATEVER ELSE FITS IN THE REMAINING SPACE. ONCE ALL ROWS AND COLUMNS ARE COMPLETELY FILLED, THE FLOW ROUTES TO THE BOTTOM-RIGHT CORNER TO OUTPUT THE MAXIMUM OPTIMAL SURVIVAL VALUE OF 1300 AND ENDS THE PROGRAM.

0/1 KNAPSACK PSEUDOCODE

```
FUNCTION findMaxPriority(weights, values, capacity, n)
    grid = 2D array of size [n + 1][capacity + 1] filled with 0

    FOR item FROM 1 TO n
        FOR weight_limit FROM 1 TO capacity

            IF weights[item - 1] <= weight_limit THEN

                leave_it = grid[item - 1][weight_limit]
                take_it = values[item - 1] + grid[item - 1][weight_limit -
weights[item - 1]]
                grid[item][weight_limit] = MAX(leave_it, take_it)

            ELSE
            grid[item][weight_limit] = grid[item - 1][weight_limit]
            END IF

        END FOR
    END FOR

    RETURN grid[n][capacity]
END FUNCTION
```

It first creates a 2D grid to act as a memory bank, which stops the computer from wasting time on redundant calculations. Nested FOR loops then scan across the grid to evaluate each item's opportunity cost by comparing two paths: bypassing the item (leave_it) versus packing it and adding its value to the best combinations that fit into the remaining weight space (take_it). By saving only the maximum of these two paths into each cell, the algorithm constantly builds on its best choices so that the final RETURN statement effortlessly grabs the absolute highest total priority score from the very last corner of the grid.

CORRECTNESS ANALYSIS

DP[I][W] = MAXIMUM SURVIVAL PRIORITY VALUE
USING THE FIRST I CRATES WITHIN WEIGHT LIMIT W.

BASE CASES:
DP[0][W] = 0
DP[I][0] = 0

RECURRENCE:

IF WI > W:
DP[I][W] = DP[I - 1][W]

OTHERWISE:
DP[I][W] =
MAX(
    DP[I - 1][W],
    VI + DP[I - 1][W - WI]
)

EVERY CRATE HAS ONLY TWO VALID DECISIONS:

1. LEAVE THE CRATE BEHIND
2. LOAD THE CRATE ONCE

THE ALGORITHM COMPARES BOTH CHOICES AND STORES THE BETTER SURVIVAL PRIORITY VALUE.

TIME & SPACE COMPLEXITY

TIME COMPLEXITY: Θ(N × W)

- THE OUTER LOOP PROCESSES N SUPPLY CRATES.
- THE INNER LOOP CHECKS EVERY CAPACITY FROM 1 TO W.
- EACH DP STATE TAKES CONSTANT TIME TO UPDATE.

BEST CASE: Θ(N × W)
AVERAGE CASE: Θ(N × W)
WORST CASE: Θ(N × W)

FOR OUR SCENARIO:
N = 6 CRATES
W = 1000 KG

CORE DP UPDATES = 6 × 1000 = 6,000

SPACE COMPLEXITY: Θ(N × W)

THE DP TABLE HAS:

(N + 1) × (W + 1)

FOR OUR PROJECT:

(6 + 1) × (1000 + 1)
= 7,007 TABLE CELLS

GROWTH OF FUNCTION

Number of Crates, n	Capacity, W	DP State Updates, n × W
6	500 kg	**3,000**
6	1,000 kg	**6,000**
6	2,000 kg	**12,000**
12	1,000 kg	**12,000**
12	2,000 kg	**24,000**

Doubling n or W approximately doubles the work; doubling both approximately quadruples it.

OBSERVATION:

- IF N DOUBLES, THE RUNNING TIME APPROXIMATELY DOUBLES.
- IF W DOUBLES, THE RUNNING TIME APPROXIMATELY DOUBLES.
- IF BOTH N AND W DOUBLE, THE WORK APPROXIMATELY QUADRUPLES.

THEREFORE, THE ALGORITHM GROWS LINEARLY WITH BOTHTHE NUMBER OF CRATES AND THE PAYLOAD CAPACITY.

PROGRAM TESTING RESULTS

Optimal Crates Selected:

- High-Yield Water Purifiers (500 kg, 650)
- Concentrated Emergency Rations (500 kg, 650)

Dynamic Programming successfully found the optimal solution.
Full helicopter capacity was utilized.
Maximum survival priority value achieved.

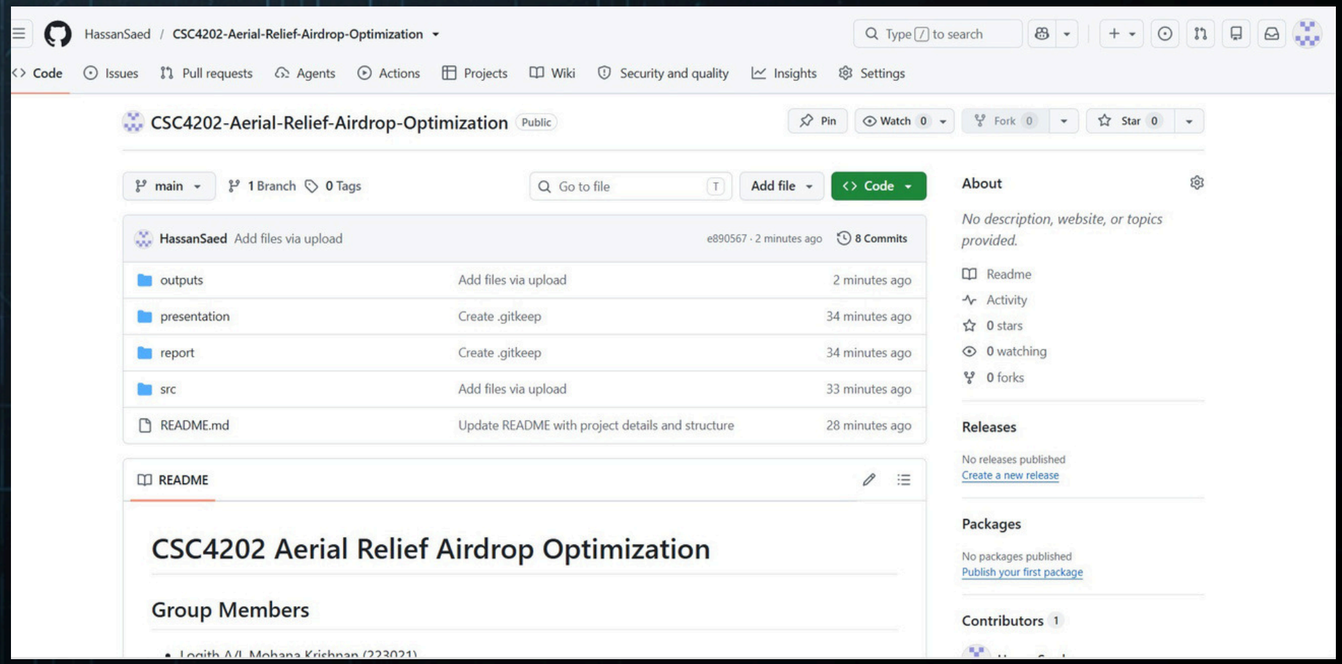
```
PS C:\Users\hassan saed> java AerialReliefOptimization
=== Aerial Relief Airdrop Optimization ===
Maximum Survival Priority Value Achieved: 1300
Total Payload Weight Used: 1000 kg / 1000 kg

Selected Supply Crates to Load:
---------------------------------------------------------------
Item Index 1 | High-Yield Water Purifiers       | Weight: 500 kg | Value: 650
Item Index 2 | Concentrated Emergency Rations   | Weight: 500 kg | Value: 650
---------------------------------------------------------------
Execution Time (Core Algorithm): 967599 ns (0.9676 ms)
PS C:\Users\hassan saed>
```

PROGRAM TESTING RESULTS

Parameter	Result
Maximum Priority Value	1300
Total Payload Weight	1000 kg
Capacity Utilization	100%
Execution Time	0.9676 ms
Status	Passed

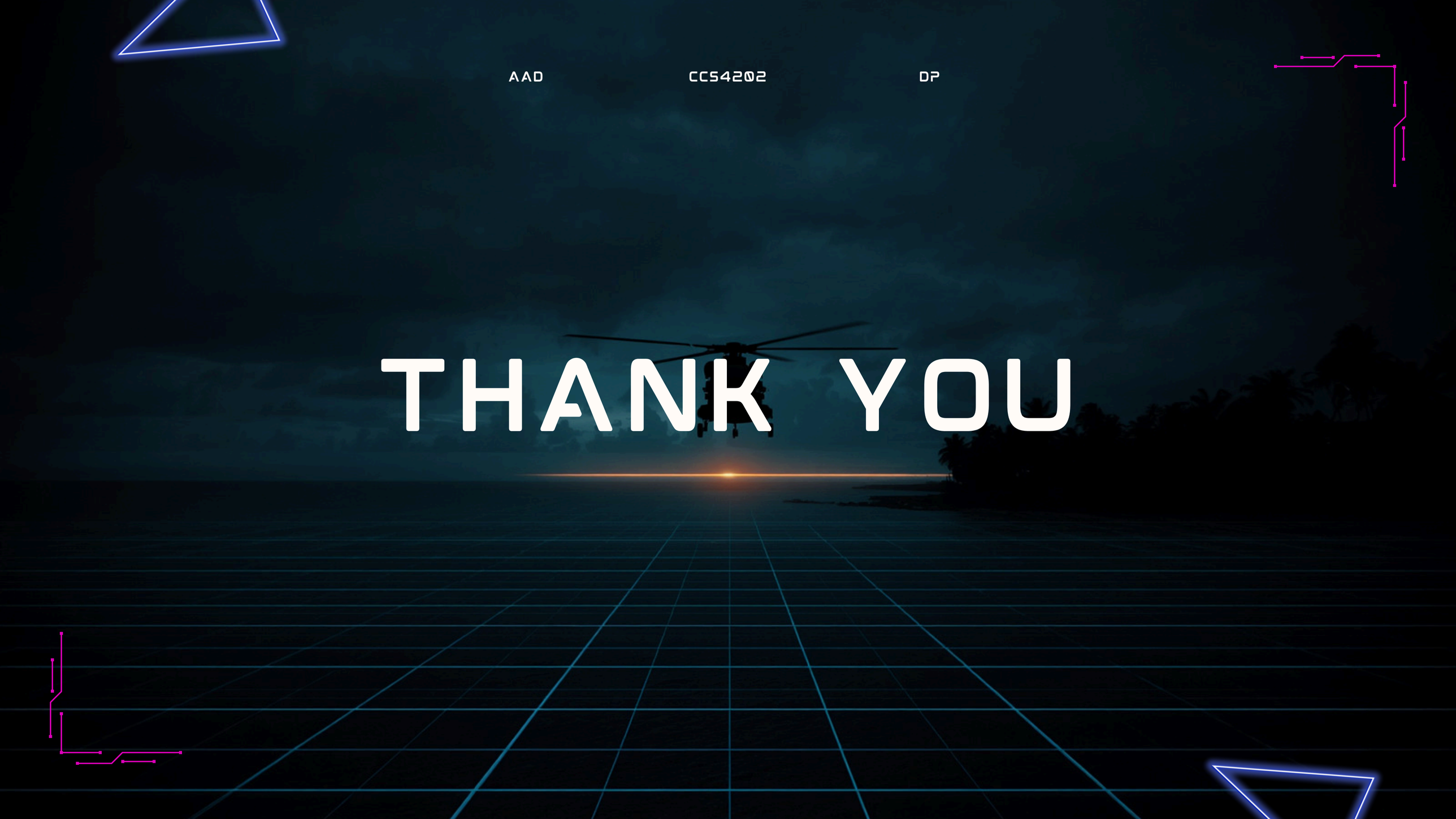
QA & INTEGRATION

- We created and managed the project GitHub repository.
- Organized project files into:
  - src
  - report
  - presentation
  - outputs
- Maintained project version control.
- Integrated source code, documentation, and testing outputs.



CONCLUSION

- Dynamic Programming successfully solved the 0/1 Knapsack optimization problem.
- The algorithm achieved the maximum survival priority value of 1300 while staying within the 1000 kg helicopter payload limit.
- Compared with the Greedy approach, Dynamic Programming guaranteed the optimal crate selection and eliminated wasted payload capacity.
- The Java implementation and testing confirmed that the proposed solution is accurate, efficient, and suitable for real-world disaster relief planning.


THANK YOU